Rapport Programmation Système Avancée

Le binôme:

Michel Chen 22202655
Qian En MA 22202681

La structure d'un bloc (hache_entree)

Notre structure est basée sur celle proposée dans le sujet, dans lequel on a rajouté deux attributs qui permettent d'optimiser et de simplifier le code.

```
struct hache_entree {
        uint32_t premier;
        uint32_t precedent;
        uint32_t suivant;
        uint32_t longueur;
        uint32_t type;
        char obj[1004];
};
```

- L'attribut **precedent** transforme la table en une liste doublement chainée. Ce qui permet d'optimiser certaines fonctions, en passant de la fin au début. L'ajout de précédent simplifie aussi énormément le fonctionnement de ***deplacer_bloc*** et nous évite la corruption des pointeurs lorsque nous déplaçons les blocs.

- L'attribut **type** permet d'identifier le type de la donnée (chaîne de caractères, fichier, nom fichier et bien plus si nous voulons rajouter d'autre sortes de type). Elle est ainsi primordiale lorsque nous manipulons les données, c'est-à-dire la recherche, l'insertion et la suppression.

Par ailleurs, la taille de l'objet a diminué par l'ajout de ces deux attributs (4 octets chacun). Cela permet d'avoir une structure à 1 ko soit 1024 octets comme spécifiée sur le sujet (1004 = 1024 - 5 * 4).

La gestion des collisions

Pour résoudre les collisions, nous utilisons le sondage linéaire (soit la fonction ***bloc_libre***) proposé par le sujet, pour localiser le prochain bloc libre en cas de conflit. Nous avons également identifié qu'il existe deux types de collisions qui se traitent différemment :

1. La collision classique: Le bloc est occupé et son hash correspond au bloc.
    - Le bloc 200 est occupé par A et h(A) = 200.
    - On ajoute B avec h(B) = 200. **Collision détectée !**
    - Le bloc B fait un sondage linéaire et trouve le bloc 201 libre.

- Le suivant de A est 201 et le premier de B est BLOC_LIBRE, soit FFFFFFFFUL.

2. Le squatteur: Le bloc est occupé mais son hash ne correspond pas au bloc.
   - Le bloc 200, 201, 203 est occupé par A avec h(A) = 200.
   - On ajoute B avec h(B) = 201. **Squatteur détecté !**
   - Le bloc B fait un sondage linéaire et trouve le bloc 205 libre.
   - Le premier du bloc 201 est 205 et le suivant ne change pas afin d'éviter de corrompre A. Et pour le bloc 205, premier, suivant vaut BLOC_LIBRE.

Les différents objets dans la table

Nous pouvons insérer trois différents types de données dans notre table. Nous aurions pu ajouter d'avantages de types, cependant nous nous sommes concentrés sur l'essentiel et ce qui nous était demandé pour les perfectionner sans problèmes au lieu de vouloir faire à moitié les choses.

1. Chaîne de caractères

L'objet stocké dans la table est bien la chaîne de caractère en elle-même. Il aura juste à vérifier qu'on ait assez de bloc pour stocker toute la chaîne de caractères puis faire l'appel à ***handle_data*** qui va s'occuper d'insérer l'objet dans la table.

2. Fichiers

Les fichiers sont insérés dans la table en chaîne de caractères sous le format métadonnée (nom + taille + date de modification + droit) + contenu. Étant donnée qu'elle est considérée comme une chaîne de caractères, on peut donc ainsi appeler la fonction ***handle_data*** qui est une fonction utilisée par *insertstring*. Par ailleurs, nous avons exclu atime (le dernier accès au fichier) car on aura alors toujours un nouveau fichier alors qu'on a pas changé le contenu.

3. Nom de fichier

Lorsqu'on insère un fichier, nous ajoutons dans la table aussi le nom du fichier qui contient l'emplacement de toutes les versions de fichiers. Cela est notamment très utile pour la fonction ***lookup***, nous avons donc accès à toutes les versions du fichier uniquement à partir de son nom. Le nom du fichier est sous la forme "nom_fichier\0index", cela nous permet donc de vérifier si c'est bien le nom du fichier est corrompu ou non, et ensuite de vérifier les index de toutes les versions qui prennent chacune 4 octets (dû au fait que premier est de type uint32_t).

Les commandes

1. **make**

Pour commencer, nous créons la table ou supprime son contenu s'il existe déjà, ensuite nous projetons en mémoire le fichier pour pouvoir initialiser tous les blocs.

2. **free**

Nous devons parcourir toute la table pour compter le nombre de bloc libre, de fichiers et de chaîne de caractères présent dans la table, puis ensuite nous affichons sur le terminal toutes ces informations.

3. **insertstring**

Pour éviter des insertions partielles puis nettoyage, il fallait vérifier si la table contient assez de bloc pour insérer toute la chaîne de caractères. Ensuite, si nous avons assez de place, il faut gérer les collisions si y'en a (via le sondage linéaire). Puis, nous pouvons enfin ajouter la chaîne de caractères dans la table.

4. **insertfile**

En considérant un fichier comme une chaîne de caractères en le convertissant en métadonnée + contenu, cela nous permet de nous faciliter la tâche. En effet, nous devons d'abord récupérer les métadonnées du fichier à l'aide de ***fstat***, ensuite on créer une chaîne qui contient les métadonnées et le contenu, qu'on envoie dans la même fonction appeler par ***insertstring*** soit ***handle_data***. Lorsque le fichier est dans la table, il faut aussi y mettre le nom du fichier.
Si le fichier n'existe pas, dans ce cas nous le créons et nous mettons l'index du bloc du fichier après le nom du fichier.
Si le fichier existe, alors nous rajoutons à la fin de l'objet l'index de la nouvelle version.

5. **lookup**

Nous cherchons l'argument donné à la fois comme chaîne de caractères et comme nom de fichier. Pour une chaîne, nous calculons son hash; puis nous parcourons la liste chaînée correspondante jusqu'à trouver une entrée **TYPE_CHAINE** avec le même contenu. Pour un fichier, nous utilisons les entrées **TYPE_NOM_FICHIER.** Elles contiennent le nom du fichier; puis une liste d'index vers les différentes versions stockées dans la table. Si un nom est trouvé, lookup affiche les versions disponibles ainsi que leurs métadonnées. L'utilisateur peut ensuite choisir de simplement être notifié, d'afficher le contenu du fichier, ou de sauvegarder une version. Dans ces deux derniers cas, le fichier est reconstruit à partir de sa chaîne de blocs.

6. **deletehash**

Nous récupérons le hash en argument, puis supprimons toutes les données qui ont ce hash, tout en gérant les versions des fichiers.
On vérifie d'abord si l'index cible est libre ou si un squatteur dont le premier vaut **BLOC_LIBRE** est présent. Si c'est le cas dans ce cas, nous envoyons un message d'avertissement à l'utilisateur, l'informant qu'aucune donnée avec ce hash est présente dans la table.
Si nous avons bien une donnée qui contient ce hash, dans ce cas, on parcourt tous les suivants jusqu'à tomber sur **FIN_CHAINE**. Par ailleurs, lors du parcours si nous

avons un type **NOM_FICHIER**, nous supprimons toutes ses versions et si nous avons un type **FICHIER** et qu'il n'y a qu'une version, nous supprimons le nom fichier. Une fois la fin de la chaîne atteinte, nous détruisons les blocs de la fin au début (***supprimer_par_derriere***). Cela nous permet ainsi d'avoir une table cohérente même en cas d'interruption, évitant ainsi des pointeurs orphelins.
Une fois que la suppression est terminée, nous utilisons la fonction **deplacer_bloc** pour reboucher les trous potentiellement formée par la suppression de certains blocs.

7. **delete**

Pour une chaîne, nous retrouvons le bloc de départ avec recherche_bloc, puis nous supprimons toute la chaîne de blocs associée. Pour un nom de fichier, nous récupérons d'abord toutes les versions du fichier, puis nous supprimons chacune des versions pointées. La suppression se fait de la fin vers le début. Cela permet de libérer tous les blocs d'un objet sans perdre les liens nécessaires au parcours. Après la suppression, nous utilisons deplacer_bloc pour déplacer certains blocs si nécessaire et garder une table cohérente après les collisions.

8. **check**

Pour vérifier l'intégrité et la non corruption de toutes les données de la table, nous devons parcourir toute la table et vérifier si le hash correspond bien à l'index dans lequel se situe la donnée.
Selon le type de la donnée, nous allons procéder à différentes vérifications afin de nous assurer que la donnée est non corrompue. Par exemple, si nous avons un type NOM_FICHIER, dans ce cas on vérifiera si toutes les versions sont bien à l'index indiquée.

9. **extend/shrink/ajust**

Ces commandes utilisent les mêmes fonctions mais ont des fonctionnalités différentes. Selon le taux de remplissage, ***ajust*** ira soit ***extend*** soit ***shrink*** la table. Ces commandes créeront une nouvelle table temporaire qui sera doublée si on ***extend***, divisé par deux si on ***shrink***. Dans les deux cas, nous devons recalculer le hash de toutes les données présentes sur la table d'origine puis les insérer dans la nouvelle table. Une fois cela fait, on supprime l'ancien fichier et on renomme la table temporaire par le nom de l'ancien fichier, cela donne l'illusion qu'on a magiquement doubler ou diviser par deux la taille de la table. Afin de pouvoir réaliser le redimensionnement plus rapidement, nous avons utilisé des threads pour insérer en même temps plusieurs données à la fois au lieu de faire un par un. Le tout est synchronisé via l'utilisation de mutex assurant l'absence de race conditions.

La concurrence

La concurrence était une partie dans laquelle nous devions modifier un peu toutes les parties du code pour ajouter le verrouillage, déverrouillage et l'attente de

slowns nanosecondes. Pour cela, il était primordial de créer trois fonctions qui nous permettraient d'éviter que le code soit redondant qui sont ***verrouiller_bloc***, ***attendre*** et ***deverrouiller_bloc***. Il faudra ensuite pour chaque accès à la table mettre un verrou sur les blocs pour éviter que deux processus puissent intervenir sur le même bloc en même temps. De ce fait, lorsque nous voulons lire un bloc, il est nécessaire de mettre un verrou de lecture et pour l'écriture, un verrou d'écriture. La fonction ***attendre*** permet justement de tester si le code fonctionne lorsque plusieurs processus utilisent la table. Par ailleurs, nous avons utilisé un verrou global dans la fonction ***make*** mais aussi dans la fonction ***extend/shrink/ajust***, car cela nous semblait absolument nécessaire. En effet, la fonction fait un appel à ***make***, ce qui crée bien un verrou global mais qui est ensuite détruit lorsque la fonction termine, ainsi nous obligeant à mettre aussi un verrou global sur cette fonction.

Le multithread

Nous avons implémenté la commande ***insertgrosfichier*** qui insère chaque ligne d'un gros fichier texte comme une chaîne de caractères indépendante. Le fichier est d'abord ouvert, puis projeté en mémoire avec mmap.

Avant l'insertion, nous parcourons le fichier une première fois pour estimer le nombre de blocs nécessaires. Cela permet de vérifier que la table contient assez d'espace libre avant de commencer. Cette vérification rend la commande plus sûre, car elle évite une insertion partielle, mais elle peut aussi être très lente, puisqu'il faut parcourir tout le fichier avant de lancer l'insertion.

Le fichier est découpé en plusieurs zones, et nous créons un thread par processeur disponible avec sysconf(*SC*_NPROCESSORS_ONLN). Chaque thread traite une zone différente. Pour éviter les doublons lorsqu'une ligne traverse la frontière entre deux zones, un thread qui ne commence pas au début du fichier avance jusqu'au prochain début de ligne.

Chaque ligne trouvée est copiée temporairement, puis insérée dans la table avec handle_data. L'insertion est protégée par un mutex afin d'éviter que plusieurs threads modifient la table en même temps.

En pratique, nous avons remarqué que les performances varient fortement selon la machine utilisée. Par exemple, les temps d'exécution obtenus sur nos deux ordinateurs et sur lulu sont très différents.